TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

Part 2

Preparing the shared model: objects become bones

The one real difference between this branch and the others: TSW wants a *skinned skeletal mesh*, not a hierarchy of objects. This part shows what the skeleton must look like, how the TSC hierarchy maps onto it, the sockets and collision the editor expects, materials and units, and the export settings. The add-on (Part 7) does all of it on a temporary copy; read this to know what it did and how to tag the few things it cannot guess.

In this part

1. Objects → bones: the conversion in one picture
2. Skeleton conventions
3. Which objects become bones
4. Weights: one bone per part
5. Sockets
6. Buffers, couplers and the physics box
7. Collision: the UCX mesh
8. The cab as a second mesh
9. Materials and textures for Unreal
10. Units, axes and handedness
11. LODs
12. Export settings, by hand and with the add-on
13. Checklist

Skeleton and socket conventions come from `SK_ART_T_Ext` as read in the editor (**verified**). The conversion, weighting, socket placement and FBX round-trip were tested headless in Blender 4.2 on a synthetic locomotive (**tested**); what the TSW importer does with the result is marked **check**.

1 · Objects → bones: the conversion in one picture

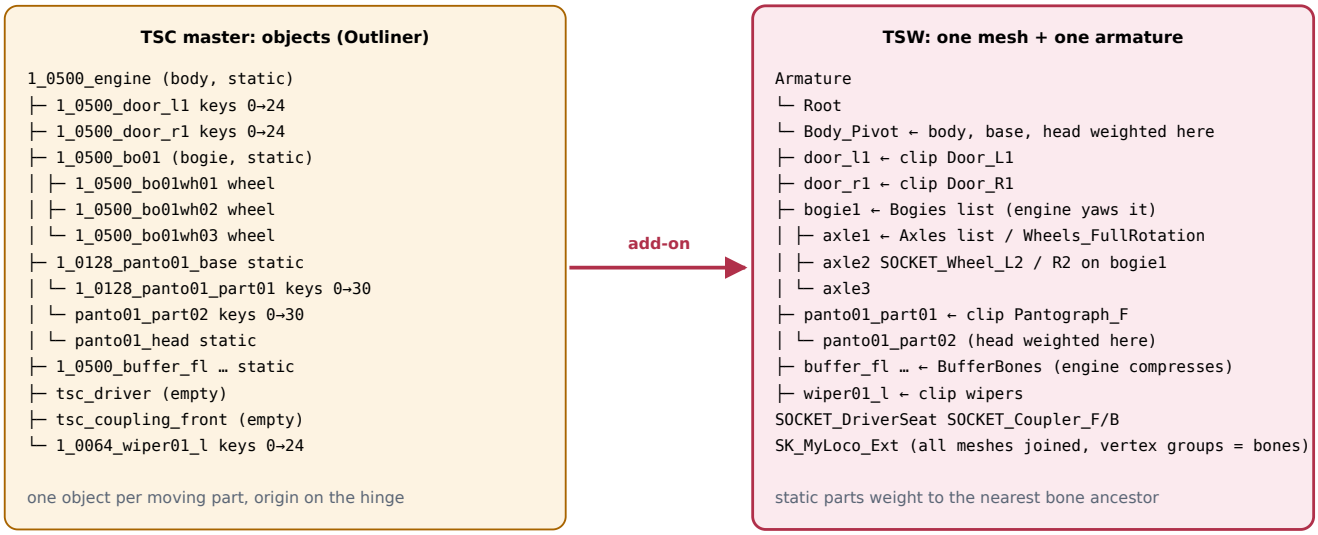


Figure 2.1 · The same hierarchy, re-expressed. Every animated or wheel/bogie/buffer object becomes a bone at its origin; static children ride on their parent's bone; everything else is welded to `Body_Pivot`. The object keys become pose-bone keys, one action per clip. Nothing in the master is changed.

2 · Skeleton conventions

Thomas's exterior skeleton, as read from the editor, is the pattern to copy: **verified**

```
Root
  Skeleton
    Body_Pivot                        whole body; sway and buffer offsets apply here
      buffer_fl  buffer_fr  buffer_bl  buffer_br
      CabDoor_L  CabDoor_R
      CabNeedle_L  CabNeedle_M  CabNeedle_R
      PistonRod_FL  PistonRod_FR
      axle1  axle2  axle3             rotate about Y (the axle axis)
      axle_Rod_L  axle_Rod_R          crank pins
      ConectingRod_FL/FR/ML/MR/BL/BR
Sockets:  Wheel_L1 Wheel_R1 Wheel_L2 Wheel_R2 Wheel_L3 Wheel_R3   (VFX at rail contact)
          DriverSeat  SecondmanSeat                                 (crew placement)
          AudioChuff  AudioRunning  AudioWhistle  AudioSteam        (RVA attach points)
```

Rule	Why
One armature, `Root` at the world origin, `Body_Pivot` as its child	The engine moves the whole vehicle by the actor; body sway and buffer offsets are applied at Body_Pivot.
Axle bones `axle1..n`, front to back, head on the axle centre line	Listed in the ABP's `Axles` so the engine can spin them; the add-on makes the bone span the wheelset.

Rule	Why
Bogie bones `bogie1..n` at the bogie pivot, axles as their children	Listed in `Bogies`; the engine yaws them to follow the track.
`buffer_fl/fr/bl/br` at each buffer head	Listed in `BufferBones`; the engine translates them by buffer compression.
Any other moving part: one bone at the object's origin, parent = nearest moving ancestor	Doors, levers, pantograph arms, wipers, needles. The clip for that part rotates this bone.
All bones world-aligned: X forward, Y left, Z up (roll 0, pointing +Y)	One rule for every bone; no per-bone axis puzzles. Rotations are baked into the clips, so bone orientation is cosmetic. **check** against how the `Axles` spin axis is chosen (Part 3 §3).
Bone names are free except the lists above	The VHID→clip mapping is by component and sequence name, not bone name. The add-on keeps TSC base names (`door_l1`).

A wagon's skeleton, built live on stream

The community creator's hopper used the smallest possible rig: `Root` at the origin, `Bogie1` and `Bogie2` as its children (Ctrl-P ▸ *Keep Offset*) at the bogie pivots, and `Axle1..4` as children of their bogie bone; bone height is cosmetic, so he lifted them to wheel level. Vertex groups: each wheel to its axle group, the rest of each bogie to its bogie group, the whole body to the root. Then join into one mesh and parent to the armature with empty groups. In the editor the Animation Blueprint later found “4 axles and 2 bogies” by itself from those bone names, so the `axleN` / `bogieN` convention is more than a convention: the engine looks for it. **community**

3 · Which objects become bones

Object	Becomes	How the add-on decides
Wheel objects `…bo01wh01`	bone `axleN`	Name pattern; numbered by position, front first.
Bogie objects `…bo01`	bone `bogieN`	Name pattern; axles under it become its children.
Objects with “buffer” in the name	bone `buffer_fl`…	Side from the object position.
Any object with keyframes or an NLA strip	bone named after the TSC base name	Doors, pantograph arms, wipers, levers, needles. Shares the clip name of its action/strip.
Object with `tsw_bone` property	bone with that name	Force a bone (a part the game will move later) or rename one.
Static child of a moving object (pantograph head)	no bone; weighted to the parent's bone	Walks up the hierarchy.

Object	Becomes	How the add-on decides
Body, roof, static detail	no bone; weighted to `Body_Pivot`	Everything else.
LOD 2+ twins, `shadow_`, `primarydigits`, `light_fwd/rev`, `tsw_skip`	dropped	TSC-only; UE handles LODs, shadows, lights and numbering itself.
Empties	sockets or build-sheet numbers	§5 and §6.

> **Origins on hinges, frame 0 is rest**
>
> The same two rules as TSC and TF3. A bone is placed at the object's origin, and the clip's first frame is the rest pose, so a door with its origin on the hinge and frame 0 closed converts without any manual fixing. A door keyed from frame 1 still works (frame 0 holds the first key).

4 · Weights: one bone per part

TSW vehicles are rigid assemblies, so skinning is trivial: every vertex belongs 100% to exactly one bone. The add-on gives each copied object a vertex group named after its bone (or its ancestor's bone, or `Body_Pivot`) with weight 1.0, then joins all copies into one mesh with an Armature modifier. If you rig by hand: Ctrl-P ▸ *With Empty Groups*, then assign each part's vertices to its group with weight 1. Never use automatic weights on a locomotive.

5 · Sockets

A socket is an empty named `SOCKET_<name>` parented to a bone (Object ▸ Parent ▸ Bone). The FBX exporter writes it under that bone and the UE importer turns it into a skeleton socket called `<name>`. check

Socket	Parent bone	Where it comes from
`Wheel_L1..n`, `Wheel_R1..n`	the wheel's bogie bone, else Body_Pivot	Generated from each wheel mesh: at the axle X, the wheel's outer face Y, the rail contact Z. Must not spin with the axle (sparks and spray spawn here).
`DriverSeat`	Body_Pivot	Your `tsc_driver` empty. Add a `socket_SecondmanSeat` empty for a second seat.
`Coupler_F`, `Coupler_B`	Body_Pivot	Your `tsc_coupling_front/rear` empties; the coupler view mesh attaches here.
`Lever_Throttle`, `Lever_Reverser` …	the lever's bone, else Body_Pivot	Empties named `socket_Lever_Throttle` (or `tsw_socket` property) where the hitbox should sit.
`AudioRunning`, `AudioWhistle`, `AudioChuff` …	Body_Pivot	Empties named `socket_Audio…`; the RVA attaches its audio components here.

6 · Buffers, couplers and the physics box

The physics component wants numbers, not geometry: the body box (width, height, length in metres), the physics body and centre-of-mass offsets (X along, Z up), buffer offsets and sizes, coupler pivot and receiving points (X along, Y up, metres). The add-on measures them and writes `<Name>.tsw_build.json`; Part 4 §6 is where you type them. Mass is not in Blender: take it from your TSC engine blueprint.

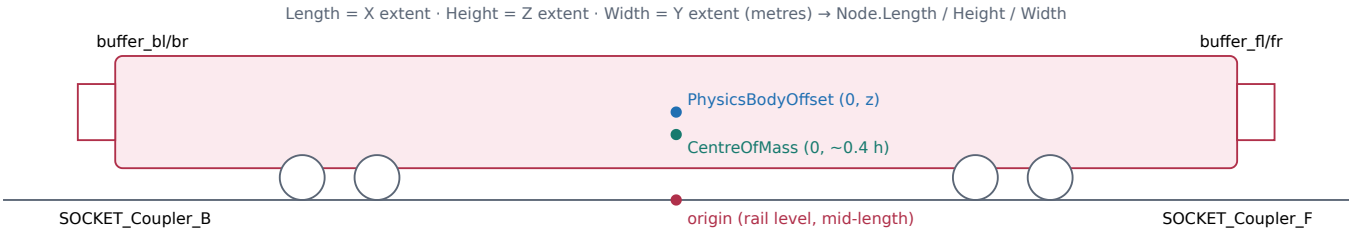


Figure 2.2 · The numbers the physics component wants, and where the add-on measures them. Keep the TSC convention: origin at rail level, mid-vehicle.

7 · Collision: the UCX mesh

Walkable collision is a separate low-poly static mesh (`UCX_MyLoco_Collision`): a box for the body plus a flat slab for the footplate or walkways. It is *not* the skeletal mesh's physics asset. Tag the objects with a `tsw_collision` property; the add-on joins them and exports a static-mesh FBX. If nothing is tagged it exports the body bounding box so the pipeline still runs. In the editor the mesh is set to use complex collision as simple. **verified** that the Class 47 and Thomas use a UCX mesh; **check** the collision setting.

8 · The cab as a second mesh

The cab interior is a second skeletal mesh (`SK_MyLoco_Cab`) on the *same skeleton*, with its own ABP. Put the cab objects in a collection and name it in the add-on's *Cab collection* field (or tag objects `tsw_cab`). Cab levers and needles are moving objects, so they become bones and their TSC control animations become clips exactly like the doors. On import, set the cab's *Skeleton* to the exterior's skeleton so both meshes share bones. **check**

9 · Materials and textures for Unreal

Unreal material input	Blender Principled input	Notes
Base Color	Base Color	sRGB. Reuse the TF3 albedo.
Normal	Normal Map	UE expects DirectX convention (green down). A Blender/TF3/glTF bake is OpenGL (green up): tick *Flip Green Channel* on the texture in UE. **check**

Unreal material input	Blender Principled input	Notes
Occlusion / Roughness / Metallic	AO, Roughness, Metallic	UE packs R=AO, G=Roughness, B=Metallic (“ORM”), linear. Roughness = 1 − TF3 gloss; the TF3 packer script can produce it.
Emissive	Emission	Lamps; driven later by LinearElectricLight parameters.

The FBX carries material *names* and base-colour textures (path mode Copy); UE creates placeholder materials on import. Build the real master material in the editor once (`M_Master`) and make instances per sheet. Keep the material count per mesh low: each is a section and a draw call.

Three habits from the stream

- Connect only the base colour in Blender; wire normal and packed maps in the editor's material editor, which is quicker for this.
- **Untick sRGB on every packed/parameter texture** (ORM, metal, roughness, AO) after import, or the PBR look is wrong. Only base colour stays sRGB.
- Moving textures and materials between folders in the Content Browser keeps every reference, so sort them into `Textures/` and `Materials/` after import. If you texture a new model, Substance Painter's “Unreal Engine 4 (Packed)” export preset writes the ORM directly.

community

10 · Units, axes and handedness

- **Units:** the editor is centimetres. Model in metres as before; export with *Apply Unit* on and *Apply Scalings = FBX All*, which writes centimetres into the file without scaling the armature. A 6 m loco imports as 600 units. check
- **Forward:** UE is +X forward. The branch rotates the TSC model (+Y forward) by −90° about Z, as the TF3 branch does.
- **Handedness:** UE is left-handed. Leave the FBX exporter's axis settings at their defaults (−Z forward, Y up); the importer's own conversion then maps Blender +X→+X, +Z→+Z and +Y→−Y. Your left-hand door stays on the left; only numbers you type by hand need Y negated.
- **Apply transforms** on the master as for the other games; the add-on copies evaluated meshes, so modifiers are baked.
- **If the vehicle imports a hundred times too big, or sits away from the origin:** transforms were not applied, or the unit scale did not reach the file. Apply all transforms, re-export with FBX All (or set the scene unit scale to 0.01 as the streamer's exporter add-on suggested), and re-import. Both symptoms happened live and both went away on re-export. community

11 · LODs

Only the LOD-1 objects are exported. Unreal generates reduced LODs for skeletal meshes in the editor (LOD Settings ▸ number of LODs, reduction per level), and the `ComponentLODManagement` component on the RVV switches off lights, Niagara and the cab mesh by distance. If you want hand-

made LODs, import them as extra LOD levels on the skeletal mesh asset from separate FBX files of your TSC LOD-2/3 twins (same skeleton). check

12 · Export settings, by hand and with the add-on

By hand (File ▸ Export ▸ FBX)

- Selected objects: armature + mesh + socket empties
- Object types: Armature, Mesh, Empty
- Transform: Scale 1.0, **Apply Scalings: FBX All**, Forward −Z, Up Y (defaults), Apply Unit ✓
- Geometry: Smoothing Face, Apply Modifiers ✓
- Armature: Primary Y, Secondary X, **Add Leaf Bones ✗**, **Only Deform Bones ✗**
- Animation: ✗ for the mesh file; ✓ for each clip file with the scene range set to the clip and the clip as the active action (the take is named after the scene)
- Path mode Copy, embed ✗
- Alternative: the free *Blender for Unreal Engine* add-on (Xavier Loux) marks an object as Skeletal Mesh, checks for “potential errors” (unit scale, unapplied transforms) and exports into an `ExportedFbx` folder next to the .blend. The streamer used it; it does not build the rig, so it suits a hand-rigged model. community

With the add-on (Part 7)

- TSC sidebar ▸ **TSW branch** ▸ Export root, Name, Cab collection
- **Preview rig in scene**: inspect bones, groups, sockets, scrub the clips, then delete the `_tsw_export` collection
- **Export TSW FBX set + build sheet**: writes `SK_Name_Ext.fbx`, `SK_Name_Cab.fbx`, `Animation/*.fbx` (one per clip, take named after the clip), `UCX_Name_Collision.fbx`, `Name.tsw_build.json`
- Report in the Text Editor as `TSW_Export`

13 · Checklist

✓	Item
☐	Origins on hinges and axle centres; frame 0 is the rest pose; transforms applied
☐	Wheels named `boNNwhNN`, bogies `boNN`; buffers have “buffer” in the name
☐	Every moving part keyed (or an NLA strip) with a clip name worth keeping (`tsw_clip` if the action name is poor)
☐	Empties: `tsc_driver`, `tsc_coupling_front/rear`, `socket_Lever_*`, `socket_Audio*`
☐	Collision objects tagged `tsw_collision`; cab objects in the cab collection
☐	Preview rig looks right: bones at hinges, clips scrub, sockets where expected

✓	Item
☐	Exported set in the plugin's folder layout; build sheet opened for Part 4